

Lab 4: Programming with Gradients

CPE 663 Special Topic III: Deep Learning

Ye Bhone Lin - 69070701715

1 Introduction

This experiment studies gradient-based optimization for insurance-charge regression, binary Iris classification, and mini-batch training. Final losses and weights are transcribed from the job output, while convergence behavior is estimated from the plots.

1.1 Slurm configuration

Table 1 summarizes the supplied batch configuration. It requests four CPUs per task, 8G of memory, one GPU, and a 30-minute time limit. These are requested resources, not measurements of utilization or elapsed runtime.

Setting	Value
Job name	lab.4
Standard output	logs/%j.out
Standard error	logs/%j.err
Partition	gpu4090
CPUs per task	4
Memory	8G
Generic resources	gpu:1
Time limit	00:30:00

Table 1: Slurm configuration.

2 Part I: Regression Model

```
(env_d1) 69070701715@cpe.kmutt.ac.th@ctl01:~/deep_learning$ sbatch slurm/lab4_part_1.sbatch
Submitted batch job 13488
(env_d1) 69070701715@cpe.kmutt.ac.th@ctl01:~/deep_learning$ cat logs/13488.out
=== Host ===
Job executed on: gpu01.slurm.cpe.kmutt.ac.th
Job started on: Tue Sep 15 03:16:35 PM +07 2026
=== Execution ===

=== Results with alpha = 0.01 ===
Final weight vector:
[-1.05266906e-16  2.25999624e-01 -9.84748600e-02  1.97325361e-01]
Iteration   0 : Loss = 0.500000
Iteration  200 : Loss = 0.445363
Iteration  400 : Loss = 0.444712
Iteration  600 : Loss = 0.444704
Iteration  800 : Loss = 0.444704
Iteration 1000 : Loss = 0.444704
Initial loss: 0.500000
Final loss: 0.444704
alpha = 0.001  final_loss = 0.4507282900075194
alpha = 0.01   final_loss = 0.44470439521049515
alpha = 0.1    final_loss = 0.44470439518957444
alpha = 0.15   final_loss = 0.44470439518957444
alpha = 0.5    final_loss = 0.4447043951895745
alpha = 1.0    final_loss = 0.4447043951895745

=== Predictions ===
   age  gender  bmi  predicted_charges
0    31      0   22         9407.691004
1    43      1   19         8188.652728
2    28      1   33        10829.298162
3    50      0   28        15517.117954
```

Figure 1: Submission of job and regression output, including final weights, learning-rate results, and predictions.

2.1 Model and batch-gradient formulation

The task specifies `insurance_charges.csv`, zero-initialized weights, and a baseline learning rate of $\alpha = 0.01$. For a linear model, let X contain the input features and an intercept column, and let $\hat{\mathbf{y}} = X\mathbf{w}$. Using the half-mean-squared-error convention gives

$$J(\mathbf{w}) = \frac{1}{2n} \|X\mathbf{w} - \mathbf{y}\|_2^2, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{n} X^\top (X\mathbf{w}_t - \mathbf{y}). \quad (1)$$

For ordinary MSE without the factor $1/2$, the gradient is twice this expression. The supplied axes say “MSE,” but the exact normalization and any feature or target scaling cannot be established without the code. Consequently, the plotted loss is not interpreted as squared charge units. Feature scaling is relevant because differently sized inputs can make gradient updates poorly conditioned.

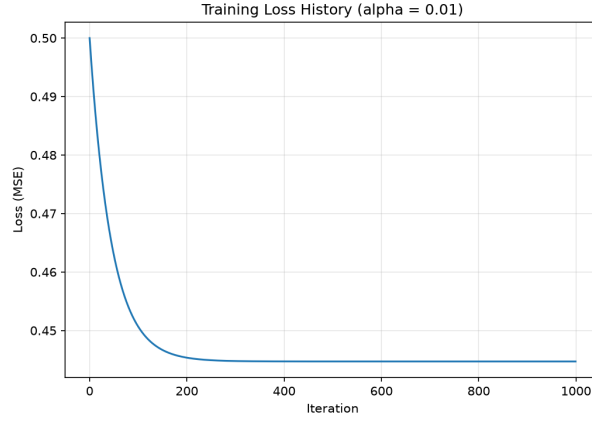


Figure 2: Training Loss History at $\alpha = 0.01$.

Figure 2 starts at 0.500000. The loss falls to approximately 0.4452 at iteration 200 and 0.4447 at iteration 800, where the curve is nearly flat. These intermediate values are plot estimates. At iteration 1,000, the logged final loss is 0.444704, an 11.1% reduction. The reported final weight vector is

$$\mathbf{w}_{\text{reg}} = [-1.05266906 \times 10^{-16} \quad 0.225999624 \quad -0.0984748600 \quad 0.197325361]^\top.$$

The vector is reproduced in the logged order; feature order and scaling should be checked against the code before using it with raw inputs. A training-loss plateau alone does not establish accuracy on unseen data.

2.2 Effect of the learning rate

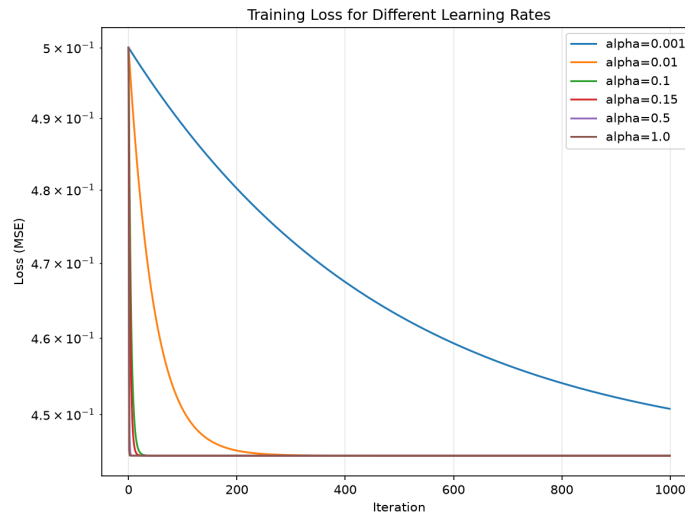


Figure 3: Regression loss for six learning rates over 1,000 iterations.

Figure 3 shows that the smallest rate, $\alpha = 0.001$, converges more slowly, reaching a final loss of 0.450728. The other five runs reach approximately 0.444704, although larger rates approach this value sooner. The final losses are rounded from the job output; their tiny differences do not indicate a practically meaningful improvement in fit.

Thus, increasing the learning rate accelerates convergence over the tested range, with no visible divergence. This does not imply that arbitrarily large rates are safe. For the quadratic objective in Equation 1, positive-curvature directions are stable when $0 < \alpha < 2/\lambda_{\max}(X^T X/n)$. The numerical limit depends on the data scaling and loss convention. Neither the largest safe rate nor a wall-clock speed advantage can be determined from these plots alone.

2.3 Predicted insurance charges

Table 2 reproduces the supplied predictions, rounded to two decimal places. Gender is retained as its supplied numeric encoding because its category mapping was not provided.

Age	Gender (encoded)	BMI	Predicted charges
31	0	22	9,407.69
43	1	19	8,188.65
28	1	33	10,829.30
50	0	28	15,517.12

Table 2: Predictions from `predictions.csv` for the four requested inputs.

The largest prediction is for the 50-year-old input and the smallest for the 43-year-old input. Because multiple attributes differ between rows, these comparisons do not isolate the effect of any one feature. No true charges were supplied for these cases, so MAE, RMSE, and other prediction-error measures cannot be calculated. The job output in Figure 1 independently confirms these predictions.

3 Part II: Classification Model

```
(env_d1) 69070701715@cpe.kmutt.ac.th@ct101:~/deep_learning$ sbatch slurm/lab4_part_2.sbatch
Submitted batch job 13490
(env_d1) 69070701715@cpe.kmutt.ac.th@ct101:~/deep_learning$ cat logs/13490.out
=== Host ===
Job executed on: gpu01.slurm.cpe.kmutt.ac.th
Job started on: Tue Sep 15 03:17:25 PM +07 2026
=== Execution ===
Final weight vector:
[-0.38529074 -0.63670144 -2.29941137 3.55917287 1.59140886]

Final training loss (BCE): 0.004332
Initial training loss (BCE): 0.693147
Iteration 0 : Loss = 0.693147
Iteration 200 : Loss = 0.036295
Iteration 400 : Loss = 0.018994
Iteration 600 : Loss = 0.013044
Iteration 800 : Loss = 0.010006
Iteration 1000 : Loss = 0.008153
Iteration 1200 : Loss = 0.006901
Iteration 1400 : Loss = 0.005995
Iteration 1600 : Loss = 0.005308
Iteration 1800 : Loss = 0.004769
Iteration 2000 : Loss = 0.004334

Confusion Matrix (rows=Actual, cols=Predicted)
          Pred: Setosa  Pred: Versicolor
Actual: Setosa         17             0
Actual: Versicolor     0             13
Accuracy: 1.0000 (100.00%)
```

Figure 4: Submission of job, final classification weights, training loss, confusion matrix, and test accuracy.

3.1 Logistic model and decision rule

The task uses `iris_2class_train.csv`, zero-initialized weights, and $\alpha = 0.1$. The logistic model and binary cross-entropy objective are

$$p_i = \frac{1}{1 + e^{-\mathbf{x}_i^T \mathbf{w}}}, \quad J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]. \quad (2)$$

The update is $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha X^T(\mathbf{p}_t - \mathbf{y})/n$. At the required threshold, predict class 1 if $p_i \geq 0.5$ and class 0 otherwise. For metric interpretation, Versicolor is positive and Setosa negative; the implementation's numeric label mapping was not supplied.

3.2 Training behavior and test results

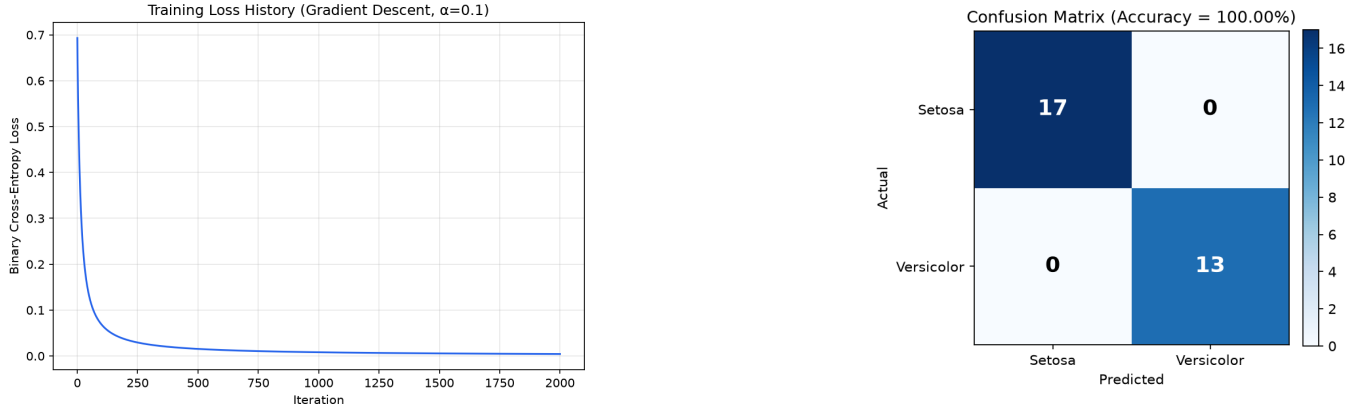


Figure 5: Supplied classification results: batch training at $\alpha = 0.1$ (left) and test confusion matrix (right).

Table 3 shows the classification loss at selected iterations. The initial and final values come from the job output; intermediate values are approximate readings of the supplied curve, not logged measurements. The loss drops rapidly at first and then improves more gradually.

Iteration	Binary cross-entropy
0	0.693147
100	≈ 0.069
250	≈ 0.030
500	≈ 0.016
1,000	≈ 0.009
1,500	≈ 0.006
2,000	0.004332

Table 3: Loss history at $\alpha = 0.1$. Intermediate losses are approximate because the per-iteration numerical log was not supplied.

The test confusion matrix (rows actual, columns predicted; class order Setosa, Versicolor) gives

$$C = \begin{pmatrix} 17 & 0 \\ 0 & 13 \end{pmatrix}, \quad \text{Accuracy} = \frac{17 + 13}{30} = 100\%.$$

All 30 examples are correct: $TP = 13$, $TN = 17$, and $FP = FN = 0$. Precision, recall, and F_1 are therefore 1.00.

In the reports the final weight vector, in the logged order,

$$\mathbf{w}_{\text{cls}} = [-0.38529074 \quad -0.63670144 \quad -2.29941137 \quad 3.55917287 \quad 1.59140886]^T.$$

This is a strong result for the supplied test split, not a guarantee of future performance. Feature order, scaling, and the original train-test split still require the source code and datasets for independent reproduction.

4 Part III: Mini-Batch Gradient Descent

4.1 Mini-batch execution

```
(env_dl) 69070701715@cpe.kmutt.ac.th@ctl01:~/deep_learning$ sbatch slurm/lab4_part_3.sbatch
Submitted batch job 13491
(env_dl) 69070701715@cpe.kmutt.ac.th@ctl01:~/deep_learning$ cat logs/13491.out
=== Host ===
Job executed on: gpu01.slurm.cpe.kmutt.ac.th
Job started on: Tue Sep 15 03:18:45 PM +07 2026
=== Execution ===
Batch size = 4   Final loss = 0.004716
Batch size = 8   Final loss = 0.008881
Batch size = 16  Final loss = 0.015255
Batch size = 32  Final loss = 0.024649
Batch size = 64  Final loss = 0.035827

=====
FINAL WEIGHT VECTORS
=====

Batch size = 4
[-0.38236248 -0.62581446 -2.25649403  3.49761605  1.55746504]

Batch size = 8
[-0.32800532 -0.53594624 -1.95898091  3.03086845  1.33809265]

Batch size = 16
[-0.28754123 -0.46488136 -1.71139874  2.64385492  1.15495941]

Batch size = 32
[-0.25049294 -0.40375937 -1.49038269  2.31335495  0.99809058]

Batch size = 64
[-0.22352676 -0.36838765 -1.33545479  2.04720922  0.88940335]

=====
TEST ACCURACY
=====
Batch size = 4   Accuracy = 1.0000 (100.00%)
Batch size = 8   Accuracy = 1.0000 (100.00%)
Batch size = 16  Accuracy = 1.0000 (100.00%)
Batch size = 32  Accuracy = 1.0000 (100.00%)
Batch size = 64  Accuracy = 1.0000 (100.00%)
  batch_size  final_loss  accuracy
0           4      0.004716      1.0
1           8      0.008881      1.0
2          16      0.015255      1.0
3          32      0.024649      1.0
4          64      0.035827      1.0
```

Figure 6: Submission of the mini-batch gradient descent job, final losses and weight vectors for all five batch sizes, and 100% test accuracy for each run.

4.2 Update strategy

For Part III, the binary classification dataset and logistic regression model from Part II were used to evaluate mini-batch gradient descent. Unlike standard batch gradient descent, which computes the gradient using the entire training dataset, mini-batch gradient descent computes the gradient using a small subset of the training data at each update.

For a mini-batch $\mathcal{B}_t$ of size b_t , the logistic gradient update is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{b_t} \sum_{i \in \mathcal{B}_t} (p_i - y_i) \mathbf{x}_i. \quad (3)$$

An epoch represents one complete pass through the training dataset. Therefore, unlike full-batch gradient descent, mini-batch gradient descent normally performs multiple parameter updates within each epoch. In this experiment, batch sizes of 4, 8, 16, 32, and 64 were evaluated over 100 epochs.

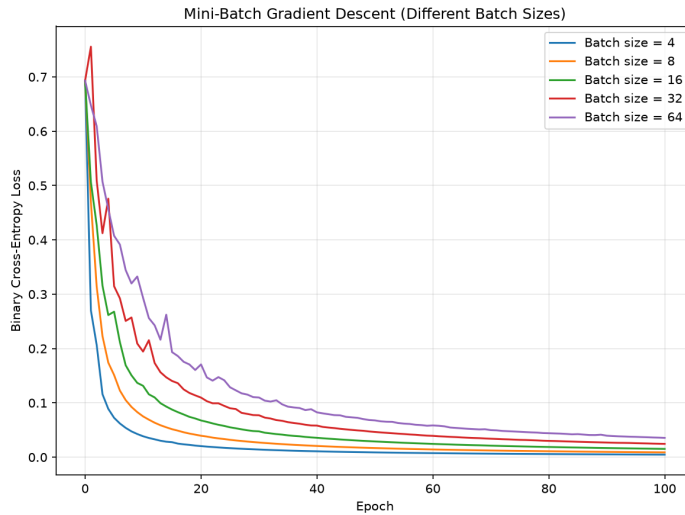


Figure 7: Mini-batch training loss trajectories for five batch sizes.

4.3 Effect of batch size

All five training-loss curves decrease substantially from an initial loss near 0.69. The final training losses and test accuracies are shown in Table 4. All five batch sizes achieve 100% test accuracy.

Batch size	Final training loss (BCE)	Test accuracy
4	0.004726	100%
8	0.008906	100%
16	0.015291	100%
32	0.024533	100%
64	0.035594	100%

Table 4: Mini-batch results after 100 epochs.

Batch size 4 achieves the lowest training loss of 0.004726, while batch size 64 achieves the highest loss of 0.035594. Therefore, within the 100-epoch training budget, smaller batch sizes achieve lower training loss in this experiment. Batch sizes 32 and 64 also show some early fluctuations before their losses decrease steadily.

One reason for the lower loss obtained by smaller batch sizes is that, for a fixed training-set size n , a batch size b produces approximately $\lceil n/b \rceil$ parameter updates per epoch. Thus, smaller batches perform more updates during the same 100 epochs. However, this comparison does not keep the total number of parameter updates or execution time constant.

Although the training losses differ between batch sizes, all five models achieve 100% test accuracy. Therefore, the lower training loss obtained by smaller batches does not result in a measurable improvement in test accuracy for this test set. Based on the training loss alone, batch size 4 performs best within the given 100-epoch budget, but execution time and additional validation would be required to determine the most practical batch size.

5 Conclusion

Regression converges smoothly at $\alpha = 0.01$; larger tested rates approach the same loss plateau sooner, while 0.001 remains above it after 1,000 iterations. The four predicted charges range from 8,188.65 to 15,517.12. Batch classification achieves a final BCE of 0.004332 and 100% accuracy on 30 test examples. Mini-batch size 4 gives the lowest training BCE (0.004726) after 100 epochs, but every tested batch size achieves 100% test accuracy. Smaller batches make more updates per epoch, so their lower losses do not establish superior runtime efficiency.